

Project Executable Files

Date	28/06/2026
Team ID	Not Assigned
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for the Rising Waters flood prediction project. All files are properly organized, named, and uploaded to the GitHub repository. The evaluator can run or deploy the solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results (k6 performance test — flood_test.js)	Yes
10	Demo video or walkthrough (if required)	NA

Step 2: File / Folder Structure

The Rising Waters project follows a clean, organized structure as shown below:

flood_project/ ← Project Root
■
■■■ app.py ← Flask web application (main file)
■■■ Procfile ← Gunicorn startup command for Render.com
■■■ requirements.txt ← Python dependencies
■■■ floods.save ← Trained XGBoost model (Joblib)
■■■ transform.save ← Fitted StandardScaler (Joblib)
■■■ flood dataset.xlsx ← Training dataset (Kaggle)

■■■ flood_eda.ipynb ← EDA & Visualization notebook
■■■ flood_model.ipynb ← Model training & evaluation notebook
■■■ flood_test.js ← k6 performance test script
■■■ README.md ← Setup and run instructions
■
■■■ templates/ ← HTML pages (Jinja2)
■ ■■■ home.html ← Landing page
■ ■■■ index.html ← Prediction input form
■ ■■■ chance.html ← Flood detected result page
■ ■■■ no_chance.html ← No flood result page
■
■■■ static/ ← Frontend assets
■■■ css/
■ ■■■ main.css ← Dark-themed responsive stylesheet
■■■ js/
■■■ main.js ← Rain animation + form validation

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://rising-waters-flood-prediction.onrender.com
Login Credentials (Demo)	No login required — application is publicly accessible without authentication
Platform / Hosting Provider	Render.com (Free Tier) — Python/Flask supported, HTTPS enforced, auto-deploy from GitHub
Repository Link	https://github.com/Gowthamsai1234/rising-waters-flood-prediction
Demo Video Link	NA — Live application accessible directly at the deployed URL above

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run the project locally:

Pre-requisites:
• Python 3.8 or above
• Anaconda Navigator / Jupyter Notebook
• Git installed on system
• Internet connection (for package installation)
Step 1: Clone the GitHub repository
git clone https://github.com/Gowthamsai1234/rising-waters-flood-prediction.git
Step 2: Navigate to the project folder
cd rising-waters-flood-prediction
Step 3: Install all required dependencies

<code>pip install -r requirements.txt</code>
Step 4: Open and run the EDA notebook
Open <code>flood_eda.ipynb</code> in Jupyter Notebook → Run All Cells
Step 5: Open and run the model training notebook
Open <code>flood_model.ipynb</code> in Jupyter Notebook → Run All Cells
This generates: <code>floods.save</code> and <code>transform.save</code>
Step 6: Start the Flask web application
<code>python app.py</code>
Step 7: Open browser and navigate to:
<code>http://127.0.0.1:5000</code>
Step 8: Access live deployed application at:
<code>https://rising-waters-flood-prediction.onrender.com</code>

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Render.com free tier sleep mode — app goes to sleep after 15 minutes of inactivity	App wakes up automatically within 30-60 seconds when accessed. No action needed — simply wait for the page to load.
2	<code>floods.save</code> and <code>transform.save</code> must be generated locally by running <code>flood_model.ipynb</code> before starting <code>app.py</code>	Resolved — both <code>.save</code> files are committed to the GitHub repository and included in the project folder.
3	No login or user authentication system implemented	Not required per project specifications. Application is intentionally publicly accessible without login for ease of use by authorities.